

IsoCompute Playbook: Optimally Scaling Sampling Compute for RL Training of LLMs

Zhoujun Cheng  * Yutao Xie  * Yuxiao Qu  * Amrith Setlur  *
 Shibo Hao  Varad Pimpalkhute  Tongtong Liang  Feng Yao 
 Zhengzhong Liu  Eric Xing   Virginia Smith  Ruslan Salakhutdinov 
 Zhiting Hu  Taylor Killian  Aviral Kumar 

* Equal contribution

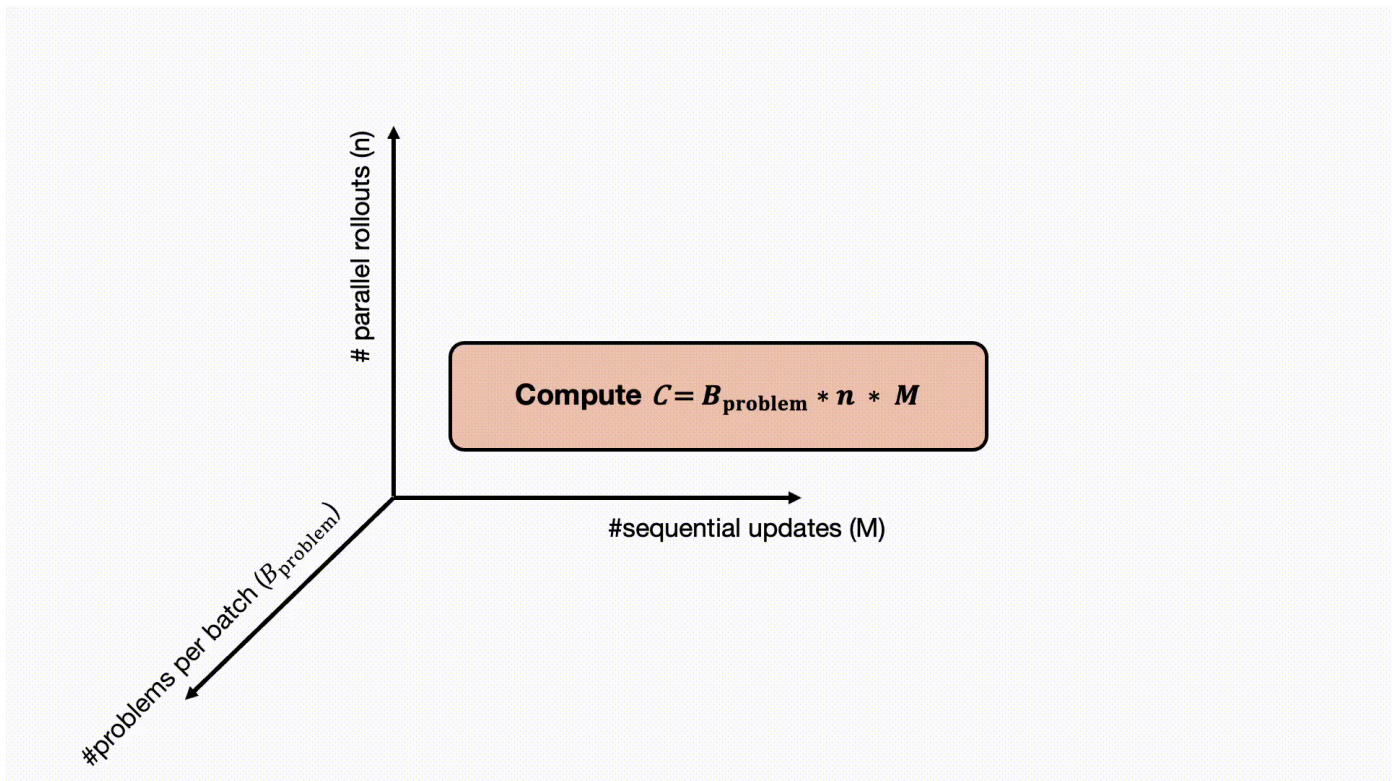


Figure 1: We study the compute-optimal RL for LLM along three axis: #parallel rollouts(n), #problems per batch(B_{problem}), and #sequential iterations(M), where total rollout compute $C = n \times B_{\text{problem}} \times M$. We find that (1) optimal parallel rollouts per problem (n) grows with compute budget (C). (2) Easy and hard problems: similar scaling trends, but different mechanisms. (3) under fixed hardware constraints ($B = B_{\text{problem}} \times n$), prioritize **large** B_{problem} (small n) at low compute budgets, but shift to **large** n (small B_{problem}) at high compute budgets to maximize performance (3) #problems per batch (B_{problem}) has marginal impact on performance when kept in a moderate range.

A persistent blocker to scaling up reinforcement learning (RL) for LLMs is the absence of a **concrete workflow**: a recipe that tells practitioners *what* to scale in the recipe, *how* to scale it, and *what outcomes of scaling one should expect*. In many areas of modern-day AI, such workflows emerge from empirical scaling laws^{1 2 3}: small-scale experiments reveal how performance should grow with compute, data, or model size. These laws inform compute allocation, models to use, and hyperparameter settings.

For RL, scaling laws remain far less understood compared to pre-training or supervised learning due to the interplay between data collection (**exploration**) and optimization (**learning from data**). Recent works have begun to sketch what these laws might look like in classical deep RL settings^{4 5 6}. But in the LLM setting, this line of work is still in its infancy. The most relevant prior results⁷ show that, under **specific conditions** (a particular problem mixture), reward curves in RL follow a clean sigmoidal shape when run for longer. Another prior work⁸ shows that RL training exhibits similar scaling behavior as pre-training in the model size, but ignores other hyperparameters. Simply fitting the shape of a learning curve or showing gains with larger models however does not answer one major question scaling laws intend to address: **resource allocation to set up an RL run under the conditions that a downstream user faces**. Given a base model, a problem set, and a total budget on compute, how should one spend this compute? If we had 10x more compute, how should that be spent? Which hyperparameters affect this resource allocation question the most and why? How does resource allocation change when the base model changes?

We have been working toward answering these questions. This article aims to frame the core scaling issues and provide practical guidance for allocating compute in on-policy RL for LLMs, with a particular focus on compute spent on **on-policy sampling (rollouts)**. The resulting picture is nuanced: scaling behavior is governed not just by total compute, but by its interaction with the base model and the prompt distribution. Nevertheless, we find that **predictable allocation rules for key hyperparameters emerge as a function of increasing sampling compute under a healthy (stable and scalable) recipe**. Concretely, for on-policy RL algorithms that optimize policies using multiple parallel rollouts per sequential gradient step, we observe the following:

- **Optimal parallel rollouts per problem (n) grows with sampling compute budget (C).** As the total sampling budget C increases, the optimal n grows following a **predictable sigmoid**. This indicates to maximize performance, practitioners should configure n based on their the **compute budget**: allocating larger n when given a higher compute budget; smaller n when the compute budget is small.
- **Easy and hard problems: similar scaling trends, but different mechanisms.** While both easy and hard problem sets benefit from scaling n , the underlying mechanisms are different. On easy problems, larger number of rollouts (n) primarily acts to sharpen the policy on already solvable problems, indicated by the improvements in the worst@k metric. In contrast, on hard problem sets, scaling n is essential for expanding coverage (discovering rare successful trajectories), directly improving the best@k metric.

- **The number of unique problems per batch (B_{problem}) impacts easy and hard problem differently.** We find that varying the number of unique problems per batch (B_{problem}) yields negligible changes in validation reward, provided it is neither too small or too large, given a fixed number of parallel rollouts (n). Thus, under fixed hardware constraints on total batch size ($B = B_{\text{problem}} \times n$), this insensitivity dictates a dynamic allocation strategy: prioritize **large B_{problem}** (small n) at low compute budgets, but shift to **large n** (small B_{problem}) at high compute budgets to maximize performance. However, the trade-off on hard problems where reward is hard to obtain is more nuanced and may be hard to predict apriori.
- **The scaling trend of parallel rollouts (n) generalizes across setups, while its saturation point is context-dependent.** We find the benefits of scaling n at higher compute hold across different base models and datasets, the specific value at which n saturates is governed by three key interactions: **a) Base model.** Intrinsic model capacities dictate the sensitivity to interference and transfer across problems, shifting the effective range of n before performance plateaus. **b) Problem set size.** Smaller datasets lower the saturation point due to **early overfitting** on easy problems, where validation reward degrades despite training reward gains, rendering very large n values infeasible. **c) Problem distribution.** Harder problems saturate at lower n values as they require a critical mass of sequential updates to make progress, effectively constraining the budget available for width. Furthermore, on skewed distributions, the scaling n trend holds, while the optimal n is heavily influenced by the specific metric one wishes to optimize (due to interference between easy and hard prompts).

▼ Where RL Scaling Laws Come From (When They Exist), Conceptually

To begin, it helps to recall why scaling laws can be derived in supervised learning or pre-training, and what makes learning dynamics predictable in these settings. In supervised learning, the objective is typically cross entropy, which is smooth and optimized over a fixed data distribution. This results in relatively well-behaved optimization dynamics, where small parameter updates lead to small and predictable changes in the loss. Consequently, under well-chosen hyperparameters, which can often be identified through small-scale experiments, the training dynamics of supervised learning become predictable, allowing the loss to be modeled as a function of compute or data. The **statistical structure of the data**, such as the covariance spectrum, remains constant through training. A simple mental model would be that the decay of this spectrum⁹ governs the power law scaling behavior observed in practice. We emphasize that this perspective is an approximation rather than a formal proof, but it provides useful intuition for why scaling laws emerge in supervised learning. In RL, the challenge of deriving any scaling law is two fold:

1. **the expected-reward objective (unlike cross-entropy) behaves non-smoothly across training iterations which make small perturbations to the parameters**
2. **the data distribution itself depends on the policy being optimized.**

When training LLMs with a binary 0/1 reward, we observe that the first challenge of non-smooth rewards is relatively mild, and that it is indeed possible to predict the expected reward. However, we did also see in our early experiments that attempting to fit other performance metrics (e.g., pass@k or worst@k) that practitioners might care about poses predictability challenges due to objective shift. For this reason, rather than fitting performance directly, we fit **compute-optimal hyperparameters** as a function of resources.

The second challenge is more fundamental. In online, on-policy RL, the base policy and every policy update determines the distribution of future experience, which is non-stationary. This in turn alters the effective data covariance seen by the policy. As training progresses, both the policy parameters and the data distribution evolve together. This violates the stationary data assumption with a fixed covariance spectrum that underlies many of the standard intuitions for scaling laws in supervised learning, and therefore the usual recipes for deriving scaling laws (e.g., in pre-training) are not directly applicable in RL.

We therefore will first seek an RL recipe whose learning dynamics scale predictably with additional compute. In online RL, performance scaling is governed by two tightly coupled mechanisms: **exploration**, which determines what data is collected, and **optimization**, which determines what and how effectively the model learns from that data. These mechanisms are inherently in tension. Exploration favors sampling unlikely but potentially informative behaviors, while optimization pushes the policy toward responses that already achieve high reward. In LLM settings, exploratory responses are often extremely low probability under the current policy, and training on them can destabilize optimization¹⁰. A common symptom of this instability is pathological behavior in the entropy of the model’s next-token distribution, which may grow uncontrollably. As a result, naive RL recipes lead to instability when run on different prompt sets. To mitigate this, we design what we refer to as a “**healthy**” **RL recipe**, in which data collection (exploration) and optimization are coupled smoothly that behavior remains stable and scaling trends can be predicted. Concretely, this entails avoiding off-policy samples, and using entropy or KL regularization only when supported by the prompt set. As we show in later sections, such recipes exhibit stable entropy dynamics and enable performance to scale in a regular manner, thereby admitting meaningful scaling-law analysis.

Given this tight coupling between data collection and optimization, small changes in exploration can substantially alter the data the model trains on next, leading to qualitatively different learning dynamics. As a result, **RL training behavior differs substantially between easy and hard problems**. We further find that repeatedly updating on the same data leads to overfitting and causes the policy distribution to shift too rapidly, breaking predictability. This effect is consistent with our prior work⁶, which shows that **clean scaling laws in RL emerge only when off-policy reuse is limited (i.e., a low updates-to-data ratio) and optimization remains stable**. Motivated by these findings, we focus on on-policy RL in the remainder of this post, and analyze scaling mechanisms separately for easy and hard problem regimes.

▼ Our Scaling Question: Compute-Optimal Sampling for LLM RL

Before we prescribe our healthy RL recipes or state our results, we first formally outline the key scaling setups that we operate in. Ultimately, scaling laws are useful because they help us decide how to allocate limited resources such as compute or data in order to achieve the best possible performance. Typically, these laws are obtained by running controlled small-scale experiments and extrapolating the observed trends. Therefore, it is helpful to begin with a basic question: **what resources actually matter in LLM RL?** In our setting, where RL is used to train a given base model on a fixed problem set, what are these resources?

In standard RL, two resources are crucial: **compute**, often measured in training FLOPs, and **data**, measured in the number of environment transitions collected online. In LLM RL, an environment naturally corresponds to a single prompt or problem, and as mentioned above, we assume access to a fixed set of such problems. When running RL post-training of a given base model on this fixed problem set, data is created entirely by spending compute on a given problem (prompt) set because every training rollout is obtained by rolling out the model (and problems are often negligibly-sized in comparison; though not always, e.g., in multi-turn settings, but we do only operate in a single-turn setting). This makes the distinction between compute and data less clear. Thus, for our study, we consider *compute to be the only fundamental resource*.

Given the primary resource of compute, it is natural to divide it into **training compute** and **sampling compute**. In LLM RL, sampling compute usually dominates the overall cost because every step of exploration requires generating multiple samples in an autoregressive manner. In addition, we typically train on all the data that is collected. Unless response lengths are scaled drastically, both training and inference compute scale (roughly) linearly with the number of tokens. This gives us a useful simplification for studying scaling. *We can treat the sampling compute budget as the main quantity to allocate*, since training compute grows in direct proportion to it, and focus on finding the most effective way to spend sampling compute. Before we go into our problem statement, we present some notation to simplify our discussion.

► Notation and Definitions

► Problem Statement

▼ What Constitutes a Healthy RL Recipe for a Given Base Model?

Now, we discuss several characteristics of an RL recipe that lead to healthy optimization behavior. From our experiments, we identify a few key factors that consistently govern whether optimization and exploration remain stable and scalable: (1) the difficulty distribution of the dataset, (2) the behavior of token entropy, and (3) the learning rate. These are not the only factors, in fact, we also observe that “staleness” (or how off-policy the generated rollouts are)

also affects scaling and performance, but leave a discussion of it from the current blog post to focus on fully on-policy approaches.

► Factor 1: Dataset Difficulty Distribution

► Factor 2: Entropy Control

► Factor 3: Learning-Rate Scaling

KEY TAKEAWAYS

1. RL training exhibits distinct training behaviors depending on problem difficulty. We therefore explicitly **curate and control for both Easy and Hard datasets** to ensure the recipe is robust to different saturation points and exploration requirements. On heterogeneous datasets that we discuss later, we simply use the recipe corresponding to the Hard dataset to avoid instability.
2. The necessity of regularization changes based on the difficulty level. **Easy tasks** benefit from KL divergence and entropy constraints to prevent premature collapse, whereas **Hard tasks** achieve peak performance when these loss functions are removed to enable stable training. Training on mixed datasets is most stable when no KL divergence or entropy are used.
3. Learning rate should not be fixed as the total batch size B changes, as in supervised learning. Of the schemes we compared, the **square-root learning rate scaling** strategy is the best.

▼ Results: Compute-Optimal Allocation of Sampling Compute in LLM RL

We now turn to our central question: *given a fixed sampling compute budget, how should we allocate it across the RL sampling knobs to maximize performance?* Recall that the rollout compute C scales $\propto B_{\text{problem}} \times n \times M$. Our goal is **not** to tune a single best configuration for training, but to identify **allocation workflows** for distributing a fixed sampling budget across problems per batch B_{problem} and rollouts n sampled for each problem in the batch. Across all experiments in this section, we sweep $(n, B_{\text{problem}}, M)$ across a range of compute budgets C . For a fixed compute budget $C = C_0$, we evaluate multiple allocations and define the **compute-optimal frontier** as the highest IID validation set reward achievable using total compute C_0 . We get different plots on the frontier as we increase and sweep over values of C_0 .

Data analysis workflow. To make our scaling law prescriptions, we subsample the data from runs to only a smaller set of **record-breaking** points on a learning curve of reward (or any other performance metric) as a function of increasing compute. A record-breaking point is defined as the first point in a run that attains a higher validation reward relative to all previous points. For

computing this higher reward, we first bucketize the reward into discrete bins and then pick the first points where the bucket increments.

► **More on the frontier definition**

We then fit a monotonic function to the record-breaking points to obtain prescriptions for compute-optimal values of n , B_{problem} , and M . Because this pre-processing is order-preserving, this procedure does not introduce spurious non-monotonicity and yields the same frontier in practice as fitting over all points.

Unless otherwise stated, we use Qwen2.5-7B-Instruct as the base model with the max output length 8,192 and the healthy *RL* recipe from above. We use on-policy updates with an Adam optimizer, scaling the learning rate proportionally to $\sqrt{B}$, where the rollout batch size $B = B_{\text{problem}} \times n$ (base LR = 1e-6 at $B = 1024$); KL and entropy regularization are enabled on the Easy set and disabled on the Hard set (as discussed above); we fix the sampling temperature to 0.6 and `topp` to 1.0 for training and evaluation; we use GRPO to estimate advantages and truncated importance sampling (TIS²⁵) to mitigate training-inference logit mismatch.

Experimental setup. We sweep over valid configurations (B_{problem}, n) from the Cartesian product $\{32 (2^5), \dots, 1024 (2^{10})\} \times \{8 (2^3), \dots, 2048 (2^{11})\}$, subject to the hardware constraint $B_{\text{problem}} \times n \leq B_{\text{max}}$. We set $B_{\text{max}} = 65,536$ for the Easy set and 16,384 for the Hard set. The value of B_{max} is smaller for the hard set to allow for more sequential iterations, within our allowed computational budget.

We concretely study compute-optimal allocation rules in three settings that allocate a subset of resources:

1. n vs M (parallel rollouts vs sequential iterations)
2. n vs B_{problem} (parallel rollouts vs the number of problems in a batch)
3. allocation across all resources.

▼ **Question 1: Trading Off Parallel Sampling n with Sequential Iterations M**

$$C \propto \underbrace{B_{\text{problem}}}_{\text{fixed}} \times \underbrace{n \times M}_{\text{user allocated}}$$

We now examine compute allocation with the number of problems B_{problem} held constant, focusing on **the trade-off between parallel samples n and sequential iterations M** under a fixed compute budget.

Fitting workflow. We plot reward vs compute C curves for each fixed n and fit a **monotonic sigmoid** to summarize how the validation set reward (avg@4) scales with

compute for that n . As mentioned above, we then define the **compute-optimal frontier** as the upper envelope of these fitted curves (see Figure 5). Then, to indicate which n lies on the frontier at each compute level, we color the frontier in Figure 5 by $n^*(C)$, which is the value of n whose fitted compute–reward curve achieves the compute-optimal frontier **up to** C . Finally, in Figure 6, we fit a log-log plot to show $n^*(C)$ as a function of C to summarize the empirical scaling behavior. We make four important observations in this setting.

1) The value of n that lies on the compute-optimal frontier shifts systematically higher as the sampling compute C increases (Figure 5). It is natural to expect larger values of n to be generally favorable at higher compute budgets (echoing BroRL²⁶), since increasing n reduces noise in advantage estimates and lowers policy-gradient variance but eats up more sampling compute. Consistent with this, the frontier-attaining $n^*(C)$ shifts to larger values as C grows, and we observe the same trend in both the Easy and Hard problem sets. Smaller values of n exhibit rapid initial gains but plateau at a relatively lower compute regime, whereas larger n sustain improvement over a broader compute range (Figure 5). This behavior also suggests that parallel and sequential compute are not exactly interchangeable. Choosing n so that we are able to perform a sufficient number of sequential updates M is necessary to achieve strong performance.

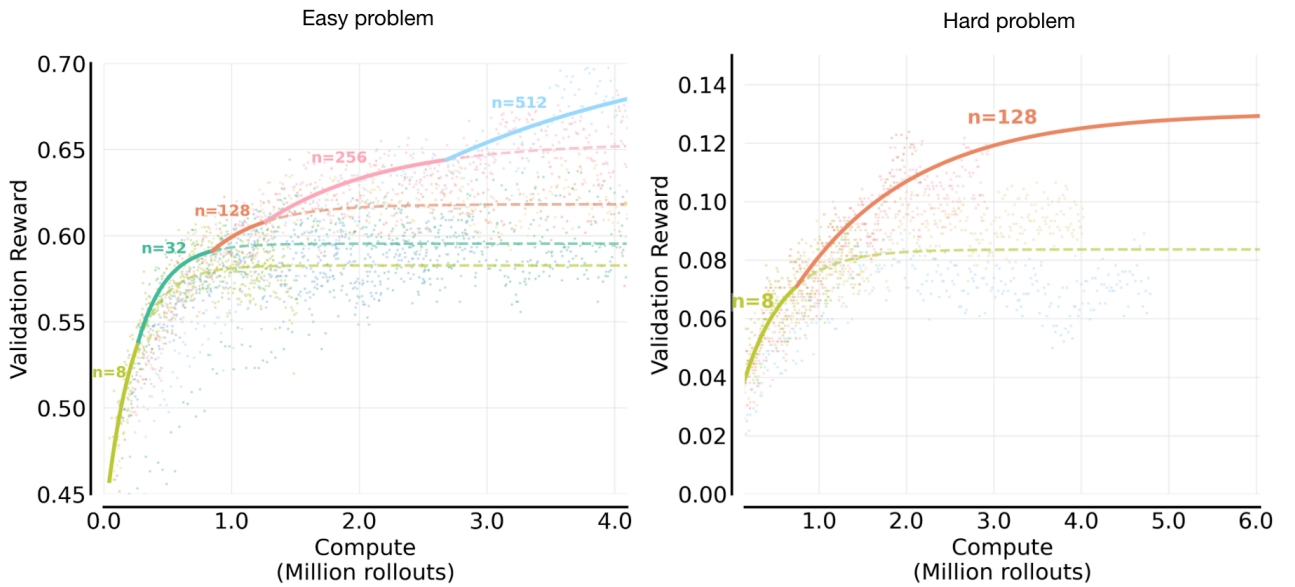


Figure 5: Validation reward frontier a function of compute ($B_{\text{problem}} = 32$), where the frontier is computed by maximizing over values of n . The compute-optimal frontier shifts to larger n with more C , showing that more parallelism becomes optimal at higher budgets. For easy problems (left), small n improves fast but plateaus; larger n sustains gains and dominates at high compute. For the hard ones* (right)*, the same trend holds, but rewards are lower and saturate earlier, with the frontier reaching a smaller value than n on the easy problems.

2) Compute-optimal values of n are well-approximated by a sigmoid function of C (Figure 6). We next seek to fit a functional relationship for the compute optimal value

$n^*(C)$ as a function of the available compute C . A natural first step is to hypothesize an appropriate functional form. As shown in Figure 5, increasing C admits larger compute optimal values of n , and over a substantial range this relationship appears approximately linear on a log-log scale. The key question is whether this growth continues indefinitely or eventually saturates. Empirically, we observe a clear saturation in Figure 6. Even when evaluating rollout values up to $n = 2,048$, values significantly larger than the saturation point, they fail to extend the frontier, with $n = 512$ continuing to dominate.

We argue that this behavior is expected for a fixed base model and a fixed problem set. To build intuition, it is helpful to view increasing n as analogous to spending more compute per gradient step. In empirical risk minimization, increasing capacity alone does not reduce validation error beyond a certain point unless additional training data is available or the train-test gap is reduced. This principle also underlies pre-training scaling rules from Chinchilla²⁷ that prescribe scaling both pre-training data and model capacity together. Perhaps most closely related to our RL training setup, our prior work²⁸ on rejection fine-tuning shows that the optimal value of n on the training set is often capped by an upper bound. Increasing n alone cannot overcome limitations imposed by a fixed problem set for training or base model. As a result, the compute optimal value of n must eventually saturate even for RL, which is precisely what we observe empirically. We empirically validate this hypothesis regarding model-data interaction in the later analysis section, where we demonstrate how the saturation point shifts given *a different base model, problem set size, and distribution*.

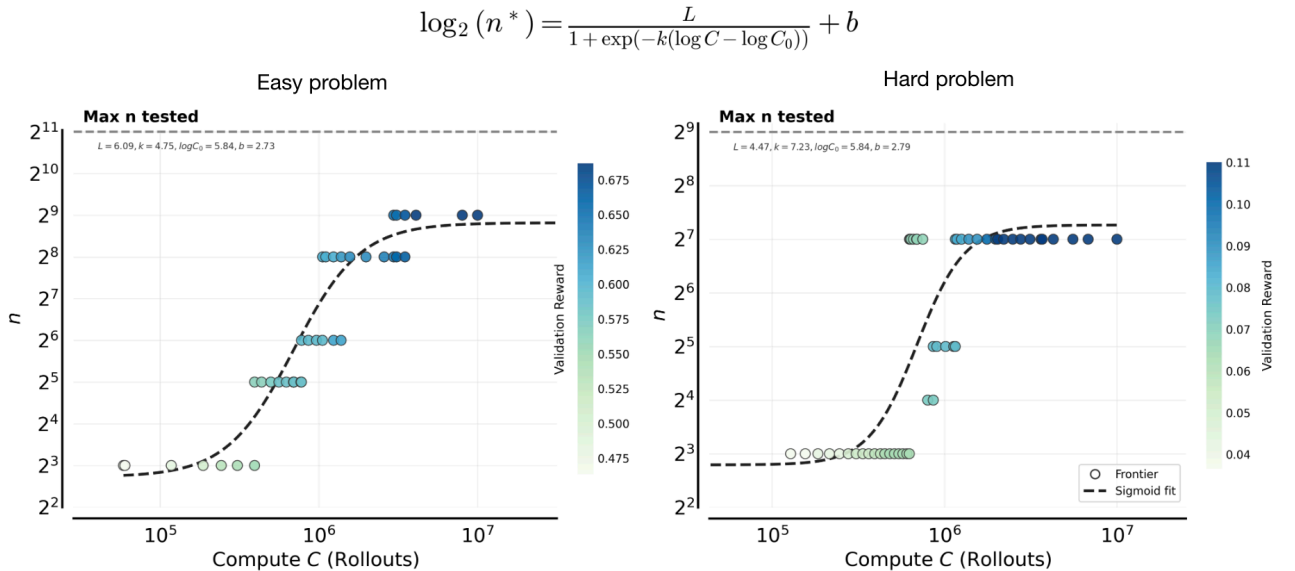


Figure 6: Compute-optimal scaling of the parallel compute n ($B_{\text{problem}} = 32$). The optimal value of rollouts n shifts systematically higher as the total sampling compute increases. Points show a running-average estimate of the frontier-attaining $n^*(C)$ at each compute budget (colored by reward), and the red curves fit a sigmoid parameterizing $\log n$ as a function of $\log C$. For both the easy set (left) and hard set (right), $n^*(C)$ rises from small to very large values as compute increases. On the hard set, the

final value converges significantly below the maximal value of n (2^{10}), and this value is lower than the easy set.

3) Next, we find that the compute-optimal allocation trend remains consistent across difficulty levels, although we find harder sets prefer smaller values of n (Figure 6). We find that the qualitative compute optimal allocation trend remains consistent across problem difficulty. On both easy and hard problem sets, the compute optimal value of n increases with total compute C before eventually plateauing. However, the plateau occurs at markedly **smaller** values of n on harder problems. In particular, very large values of n , such as $n = 512$, yield lower final performance on the hard set and do not lie on the compute optimal frontier. **This suggests that task difficulty imposes an upper bound on how large n can be used effectively.** While it may seem intuitive that harder problems should benefit from larger n due to increased sampling right away, we observe the opposite behavior in practice. On sufficiently hard problem sets, increasing n allocates substantial compute to problems where the model receives little or no learning signal. In contrast, smaller values of n focus optimization on the subset of prompts where nonzero signal is already present and meaningful improvement is possible. **Therefore, if compute is bounded**, it is better to use a smaller value of n to increase the frequency of parameter updates (small n , large M , more epochs on the same subset of problems) that exploits reachable gains, rather than large parallel compute on problems that are persistently unsolved (large n , small M , fewer training epochs).

4) Implications on optimization dynamics on the easy and hard sets and the role of various performance metrics (Figure 7). We saw in point 3 above that a smaller value of n was more preferable for optimizing validation *average reward* (avg@4 per problem), and we attributed this behavior to the underlying behavior of RL training (i.e., solving more problems vs producing more correct rollouts on the same problem). In this point, we aim to better understand this optimization dynamics and evaluate how $n^*(C)$ changes if we were to change **the target performance metric** we study.

In particular, we consider two performance metrics: **best@k** (or *pass@k*) and **worst@k**. Recall the definitions:

- **best@k**: the proportion of problems where *at least one* generated response out of k is correct. This measures the model's **coverage** over the validation problem set.
- **worst@k**: the proportion of problems where all k generated responses are correct, also referred to as *perfect solvability*. This measures the robustness of the training procedure (i.e., the degree to which it can “**sharpen**” around the right solution).

Modulo compute-optimality, a larger value of n coupled with as many sequential update steps as needed, should in principle, result in higher values for both best@k and worst@k on a training dataset. However, this is not quite the case when compute is bounded. We

empirically identify the optimal values of $n^*(C)$ for obtaining the highest best@k and worst@k scores on the validation set, across different B_{problem} values for the largest value of C , and show this number in Figure 7 below. We choose $k = 4$, much smaller than any value of n we study ($n \gg k$), so that none of the trends in Figure 7 are “edge” cases or artifacts of empirical/fitting/workflow/statistical error. Perhaps surprisingly, we now see an interesting divergence in trends of compute-optimal n that impacts the Easy and Hard sets differently.

1. On the easy set, a **larger n is compute-optimal for worst@4 (sharpening) performance, whereas relatively smaller values of n are compute-optimal for the best@4 performance**. This means that a larger n primarily improves by sharpening more on easy problems, while a smaller n suffices to sample one correct rollout (expected since the set is easy).
2. Conversely, for hard problems, a larger n is more critical for pushing the best@4 (coverage) boundary, while a relatively smaller n is compute-optimal for improving worst@4 (sharpening). However, there is a limit beyond which a larger n does not improve coverage on new problems in a compute-optimal manner, as indicated by Figure 7 that optimal values here remain generally lower than on the easy set). On the *Extremely Hard* set consisting of all pass@128=0 problems (Appendix A, Figure 21), we see a clearer tradeoff of coverage and sharpening: while larger n improves best@k, excessive n degrades worst@k and lowers the average reward. Thus, if targeting average reward, the optimal n on hard problems is the value that balances coverage and sharpening well.

The net effect of these distinct optimization dynamics is a similar trend of compute-optimal n on the validation average reward (Figures 5 & 6), but these results imply that the target performance metric itself dictates the landscape of compute-optimal n .

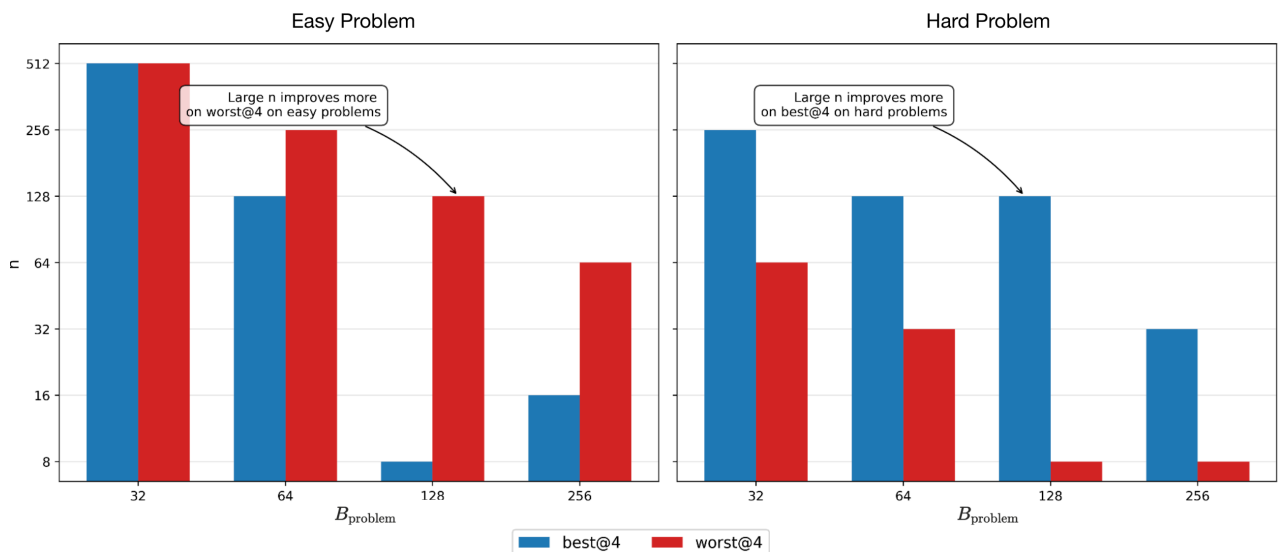


Figure 7: Different mechanisms of how n values optimize best@4 vs. worst@4 on easy and hard problems. For a given value of B_{problem} on the x-axis, the n values shown on the y-axis yield the highest validation reward under best@4 and worst@4 metrics. On the Easy set (*left*), the best value of n for best@4 (**blue**) is smaller than the best value of n for worst@4 (**red**), indicating that improving robustness (worst@4) requires substantially more parallel rollouts compared to improving coverage.

In contrast, this trend reverses on the Hard set: a larger n is needed to improve best@4 in a compute-optimal manner, while the value of worst@4 saturates at smaller n (*right*).

KEY RESULT TAKEAWAYS

1. The compute-optimal n frontier **shifts systematically higher** as the total sampling compute increases, which is well fit by a **sigmoid curve**.
2. The trend remains consistent **across training dataset difficulties**.
3. The source of gain from large n **shifts based on the training data difficulty**: scaling n improves **sharpening** (worst@4) on the Easy set, but expands **coverage** (best@4) on the Hard set.

WORKFLOW TAKEAWAYS

- Depending upon the composition of the problem set, and how effectively can the base model learn on this set, we might see different underlying mechanisms for performance improvement. It is advisable to evaluate the mode of performance improvement for your base model on your prompt set, and accordingly use it to set n as a function of the available C .

▼ Question 2: Bounded Parallel Batch Compute: Trading off B_{problem} with n

$$C \propto \underbrace{B}_{\text{fixed}} \times M = \underbrace{B_{\text{problem}} \times n}_{\text{user allocated for a fixed } B} \times M$$

Next, we study a different scaling setup, where wish to allocate a fixed total batch size B into the number of prompts used and the number of rollouts per prompt used. This question is important in practical settings where hardware parallelism (e.g., number of GPUs or data-parallel workers) is fixed, and a practitioner needs to make this compute allocation. In such cases, B is often chosen as the largest rollout batch size that saturates sampling throughput ("system batch size"). We experimented with $B_{\text{problem}} = 8, 16$ for the Easy set under fixed B to locate the upper and lower bounds for values of B_{problem} and n that are effective.

We specify the number of sequential iterations M *a priori* and seek allocations of B_{problem} and n under a fixed total batch budget $B_{\text{problem}} \times n \leq B$ that maximize performance. We observe the following:

1) On the easy problems, allocate more parallel compute n when sequential steps M is large. In this regime, we examine the compute-optimal value of n under a fixed total batch size (illustrated with $B = 8,192$ only below), as M varies. As shown in Figure 8, the optimal choice $n^*(M)$ exhibits a sigmoidal dependence on M . This behavior suggests that when more sequential update steps are available, it is generally preferable to allocate additional compute toward increasing n , rather than increasing B_{problem} . In contrast, when M is small, allocating batch size toward a larger B_{problem} is more effective, as it enables many more epochs of training on the same problems (Figure 9) within a given number of sequential updates.

On the Hard set, however, the scaling behavior is less consistent. The compute-optimal value $n^*(M)$ exhibits a non-monotonic dependence on M (see Appendix A, Figure 20), which implies a similarly irregular trend for the optimal B_{problem} . This is one of the main differences we see in scaling strategies across prompt sets of different difficulties. This happens because the Hard set behaves differently from the Easy set.

Across all settings, learning signal in RL can be increased through two complementary mechanisms: (i) **sampling**, by increasing n to raise the probability of observing correct rollouts on a given problem, and (ii) **coverage and generalization**, by increasing B_{problem} so that policy updates are informed by a broader set of problems with at least some learning signal. On the Easy set, where the base model already produces correct rollouts reliably, the dominant bottleneck is sampling quality, making larger n consistently preferable. On a harder problem set, we find that small values of n remain ineffective at extracting a gradient signal when M is small, even when it comes at the cost of training on a subset of problems in the Hard set. Next, when we increase M , contrary to the easy problem setting, we find that increasing B_{problem} is now preferred as it avoids overfitting to the small training set of problems on which we got some signal initially. And then when we increase M further we can now safely update on more problems without reducing n for a higher B_{problem} , and the compute-optimal allocation once again demands a higher value of n .

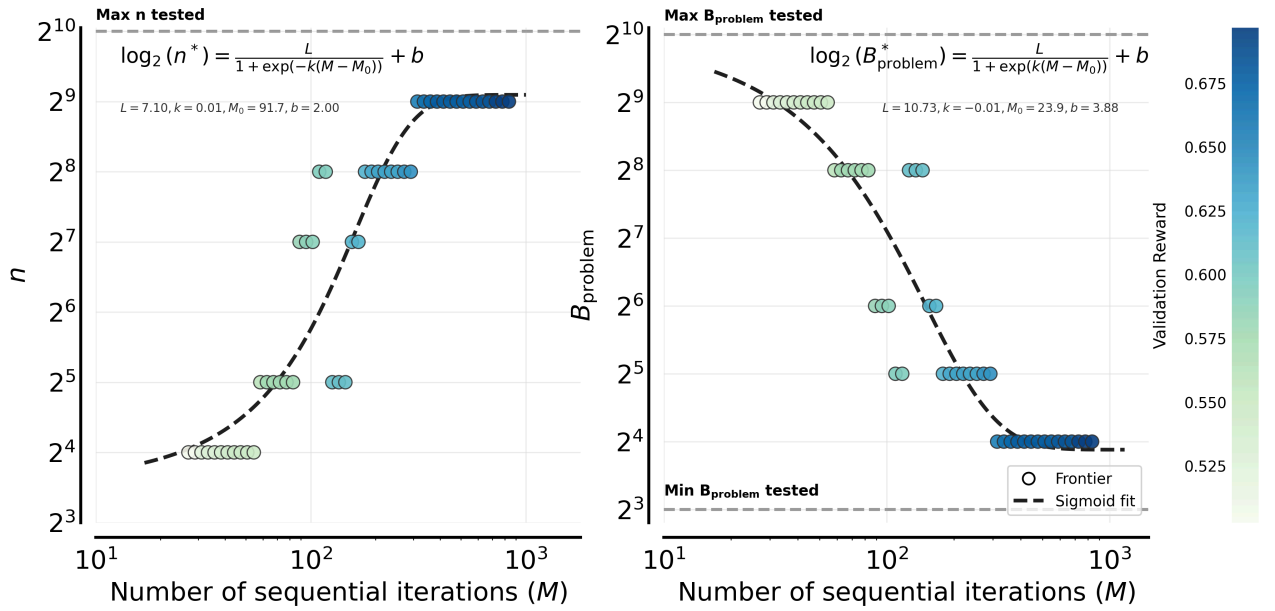


Figure 8: Compute-optimal allocation shifts from B_{problem} to n under a fixed total batch size constraint ($B = 8, 192$) on easy problems.

We fix the total rollout budget per step ($B = B_{\text{problem}} \times n$) and sweep the number of sequential iterations (M). A sigmoid relationship can explain the frontier of the optimal value of n per problem. This curve indicates that $n^*(M)$ increases with M , a proxy of total compute C given a fixed B (left). The corresponding compute-optimal number of prompts $B_{\text{problem}}^*(M)$ decreases with the available sampling compute according to an (inverse) sigmoid (right). These indicate the strategic shift toward higher per-problem sampling at larger compute budgets.

Next, **we ask what role does B_{problem} play?** To answer this question, we study the effect of modifying B_{problem} for a fixed n and the effect of modifying n for a given B_{problem} and assess if performance is more sensitive to B_{problem} or n . Our finding reveals that B_{problem} has minimal effect on validation performance.

2) B_{problem} has a marginal effect on validation performance when fixing n . As shown in Figure 9 (left column), when fixing n , varying B_{problem} on the Easy set provides little variance on validation reward. In contrast, increasing n (fixing B_{problem}) shows a strong correlation with improved validation scores, up to the saturation point discussed in Question 1. We also find a similar trend on the Hard problem set (Figure 9; right column). For the easy set, this explains the sigmoidal trend in Figure 8: since n is the primary driver of performance and B_{problem} yields little difference, increasing n is preferred for large C and B_{problem} decreases in return ($B_{\text{problem}} = B/n$). However, though note that the variation in performance from n and that from B_{problem} is overall smaller for the Hard set, which means that the optimal n is more noisy; occasionally tilting the balance towards increasing B_{problem} to attain higher performance.

Overall though, we find that setting a large n (up to the saturation point), with a moderate B_{problem} , is the most robust strategy. For example, in our experiments, we observed no significant threshold effects for B_{problem} between 32 and 1024 on the Easy set. However, on the Hard set or a skewed problem distribution, we speculate they may require a higher minimum B_{problem} for effective training. This behavior is intuitive. On a uniformly constructed Easy set (Figure 2), a relatively small but representative subset of problems already provides a good estimate of the underlying expectation. In contrast, for hard problems to the base model, training on a larger set of problems is necessary, since a small subset of problems that receive early learning signal can otherwise dominate through the course of training due to the “rich-gets-richer” dynamic of RL training. At the same time, the value of n remains important. The resulting trade-off between problem coverage and per-problem n leads to less predictable behavior on the Hard set (and explains the non-monotonic trends in setting n and B_{problem} as shown in Appendix A Figure 20 and discussed above).

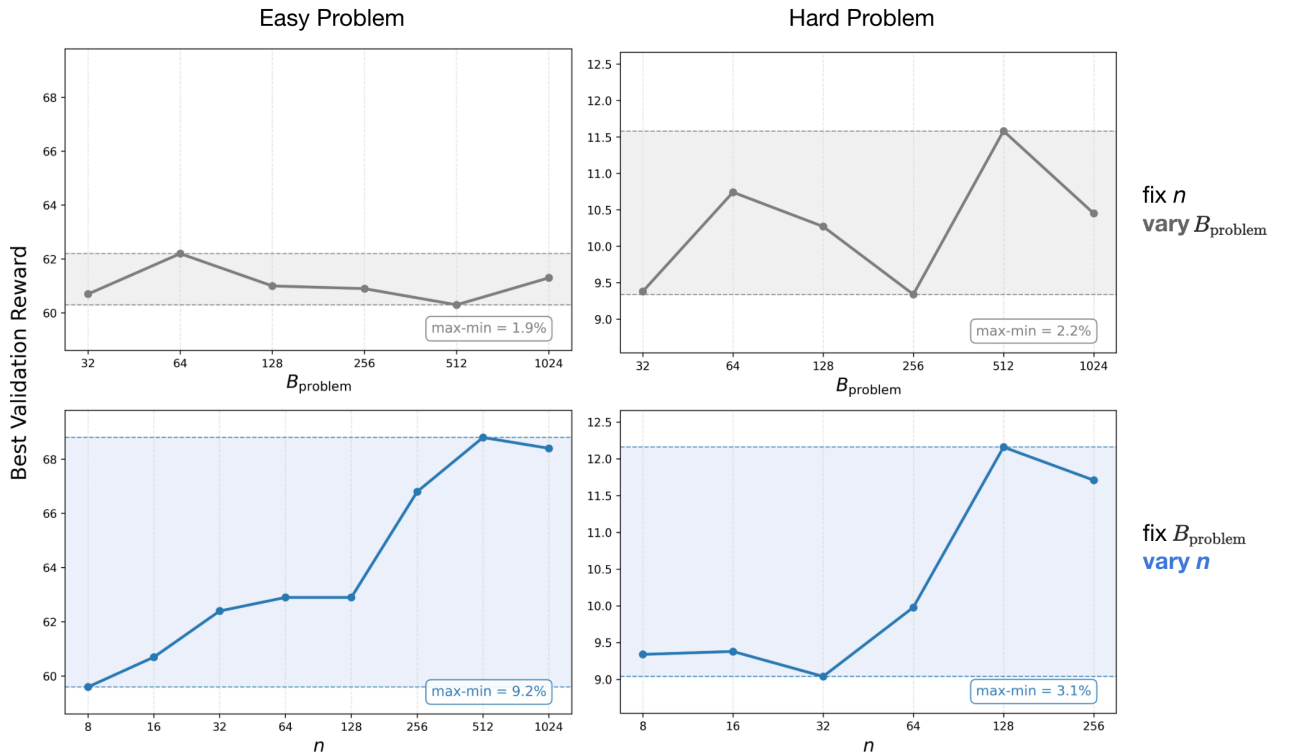


Figure 9: Sensitivity of validation reward to B_{problem} versus varying n . Top Row: Varying B_{problem} with fixed $n = 16$. Bottom Row: Varying n with fixed $B_{\text{problem}} = 32$. Left (Easy): Validation reward is driven almost entirely by n (9.2% variation; aligned with Question1 findings), while varying B_{problem} yields marginal impact (1.9% variation). Right (Hard): The trend is more nuanced. While n remains important, the sensitivity to B_{problem} (2.2% variation) becomes significant relative to the sensitivity to n (3.8% variation) given the inherently low reward regime of hard tasks. The fluctuating trend in the top-right plot suggests that B_{problem} selection introduces optimization differences on hard tasks, explaining the less predictable allocation trends when fixing B on hard problems. That said, the trend of performance with varying values of n for a given fixed B_{problem} (bottom row; Hard problems) is still monotonically increasing indicating predictability of the best value of n .

KEY RESULT TAKEAWAYS

1. With a fixed total batch size B , increasing compute favors allocating more rollouts (n) per problem and fewer problems per batch (B_{problem}). On the Easy set, this trend is clean following a sigmoid relationship, since large B_{problem} leads to rapid overfitting (as a result of multi-epoch training).
2. On the Hard set, this trend is non-monotonic: increasing B_{problem} may be desirable at some intermediate values of M . While scaling n remains the more critical resource to allocate, B_{problem} must maintain a minimum threshold to avoid incomplete optimization.

WORKFLOW TAKEAWAYS

- It is preferable to train on fewer problems with a sampling large budget n if we are allowed training for multiple epochs on the same problem set. On the other hand, if multi-epoch training is not possible, then it might be preferable to include on more problems in a batch.
- The value of minimal allowed B_{problem} is larger for the Hard set compared to the Easy set, where n and B_{problem} both lead to more comparable differences in performance.

▼ Question 3: Putting It All Together

$$C \propto \underbrace{B_{\text{problem}} \cdot n \cdot M}_{\text{varying user allocation}}$$

Finally, we relax all constraints and optimize $(B_{\text{problem}}, n, M)$ jointly under a fixed compute budget. Consistent with our previous findings, **the compute-optimal strategy is primarily defined by scaling n** . As shown in Figure 10 and 11, the optimal $n^*(C)$ follows a sigmoidal trend as compute increases, regardless of problem difficulty. In this regime, B_{problem} acts as a stability constraint rather than a performance driver. We discussed in Question 2 that setting B_{problem} within a moderate range (e.g., 32 to 1024) only yields a small variation in performance. While we did not study massive values of B where both B_{problem} and n are large, and their interaction at large batch sizes is certainly important for future work to study, we do exhaustively sample values of B_{problem} and n within the range we had access to. **Our resulting practical recipe is simple: scale n with compute according to the sigmoid, while keeping B_{problem} large enough to stabilize training.**

$$\log_2(n^*) = \frac{L}{1 + \exp(-k(\log C - \log C_0))} + b$$

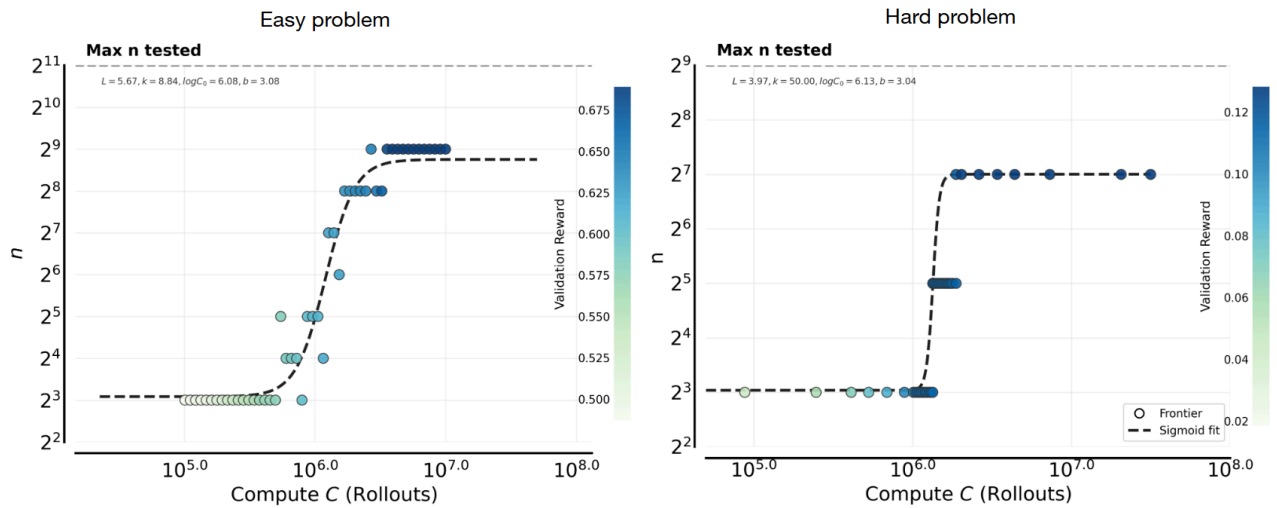


Figure 10: Compute-optimal parallel rollouts $n^*(C)$ as a function of total compute C , when B_{problem} , n , and M are all allowed to vary. We sweep hyperparameters (B_{problem}, n, M) to find the global optimal configuration at each compute budget. The compute-optimal n increases monotonically with compute, fit by a sigmoid function on both easy (left) and hard (right) problems.

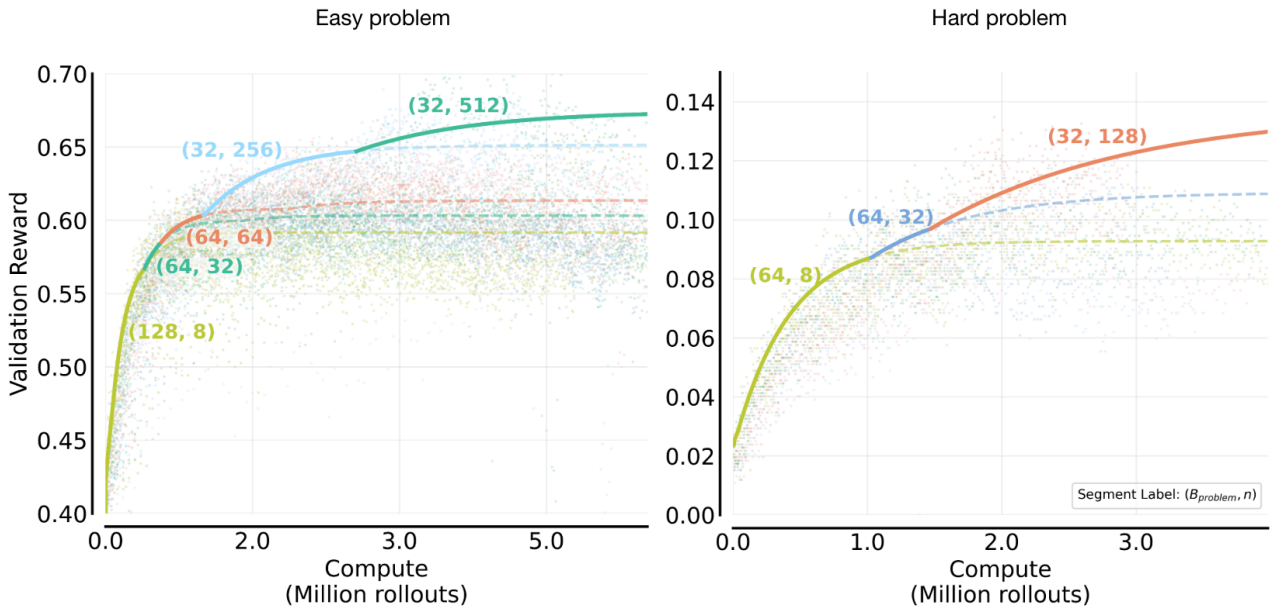


Figure 11: Compute-optimal frontiers sweeping (B_{problem}, n, M) on easy and hard problems.

We annotate (B_{problem}, n) on the frontier as they are configurations before training and M automatically scaled during training. Consistent with our earlier findings, the optimal n shifts systematically higher as compute increases. While B_{problem} changes on frontiers, its impact on performance is marginal (as discussed in Question 2 Figure 9) and thus, unlike n , the specific B_{problem} on the global frontier is more unpredictable.

KEY TAKEAWAYS

1. When jointly optimizing across all hyperparameters (n , B_{problem} , M), the compute-optimal value of n still increases with C , similar to the findings from Questions 1 & 2.
2. Note the best total rollout size B must generally increase as C increases, though the compute-optimal value of B_{problem} can be roughly chosen to be a constant at all compute budgets.
3. Our prescribed workflow suggests tuning n first for a new model or new run, followed by allocating B_{problem} to a reasonable value and setting M accordingly to the remaining compute. This provides the practical recommendation for users.

▼ The Bigger Picture: Role of the Base Model and the Prompt Set

The recurring takeaway from the analyses above is that **the compute-optimal number of rollouts, n , increases with available sampling compute C** and eventually saturates as training becomes bottlenecked by other factors. While this conclusion is straightforward, a more interesting observation is that the same qualitative trend appears on both Easy and Hard sets. This naturally raises several questions: do we expect this behavior to exist on datasets of other difficulty levels? Does it extend beyond the single base model considered so far? More broadly, under what interactions between a base model and a prompt distribution should we expect this trend to generalize? In the remainder of this section, we address these questions using a combination of conceptual arguments and empirical evidence. We begin by studying an “idealized” setting of training on a single problem, examine when predictions from the one-prompt case break down, and finally analyze how interactions of the base model and prompt set deviations from this ideal behavior.

► Base Case: Only One Training Problem

► Interference: Extending to Multiple Training Problems with Base Models

► Train-Test Gap

► Dataset Skew: Extension to Other Data Compositions

KEY TAKEAWAYS

1. **Interference:** While sequential computation is more preferred in a tabular setting, interference across problems tilts the balance in favor of parallel computation; such that both n and M should be scaled as the available compute increases.
2. **Train-test gap and overfitting:** The size of the training problem set manifests as a train–test gap: when the training set is small, validation performance saturates early.

This leads to lower saturation values for n and M , and correspondingly higher optimal values of B_{problem} .

3. **Scaling rules transfer across data compositions:** Although different base models exhibit different levels of interference on different problem sets, we observe similar scaling rules for how compute should be partitioned into parallel rollouts although for different underlying mechanisms. That said data compositions do affect the range of compute-optimal values of n .

▼ Discussion, Summary, and Future Work

A central takeaway from this work is that **healthy RL recipes are inherently dependent on the prompt distribution and the behavior of RL training depends on the interaction between the base model and the prompt set**, and that this dependence manifests directly in how optimal hyperparameters scale with compute. The same algorithm can exhibit qualitatively different scaling behavior on easy versus hard problem sets. On easier problems, increasing parallel rollout compute primarily improves sharpening and robustness, whereas on harder problems the dominant effect is expanded coverage. While trends in compute-optimal hyperparameters are often consistent when measured using average reward, they can diverge substantially under alternative metrics such as best@k and worst@k. This sensitivity to both data difficulty and evaluation metric highlights a key departure from supervised learning, where scaling behavior is typically more uniform once the model size is fixed. In RL, scaling laws are therefore inherently more nuanced, reflecting the coupled effects of optimization dynamics, exploration, task structure, and evaluation criteria. This study provides, perhaps the first, concrete framework for identifying and reasoning about these trends with base models and prompt sets, and empirically illustrates them across several regimes.

Our analysis also surfaces an important open challenge for future work: **interference across problems**. In an idealized single-problem setting, one might expect clean exponential improvements with increasing sampling compute. In practice, however, RL is performed over mixtures of problems, where progress on some tasks can interfere with learning on others. This population-level interference alters both the coefficients and effective hyperparameter values in observed scaling laws.

A promising direction is to identify sufficient statistics early in training that capture the degree of interference across problems, enabling more accurate predictions of how additional compute will translate into subsequent learning progress. We believe that tracking changes in the pass@1 distribution over the course of training provides a natural starting point for studying interference. Developing such models would be a critical step toward predictive scaling laws for RL on heterogeneous data mixtures. Mathematically, this points toward approximate closed-form rules for compute-optimal hyperparameters that generalize across base models and prompt

distributions by estimating a small number of statistics that summarize the pass@1 landscape and incorporating them into scaling-law fits. This remains an interesting direction for future work.

▼ Acknowledgements

We thank Oleg Rybkin, Apurva Gandhi, Charlie Snell, Matthew Yang, Rishabh Agarwal, Sang Michael Xie, Junlong Li, and Zora Wang for their thoughtful feedback on the blog. We also thank Chengyu Dong, Mikhail Yurochkin, Rupesh Srivastava, Joel Hestness, and Gavia Gray for early discussions on RL scaling in LLM. We also gratefully acknowledge the Orchard cluster at the FLAME center of CMU for providing some computational resources.

▼ Appendices

- ▶ **A. Additional compute-optimal results**
- ▶ **B. Generalization to OOD tasks**
- ▶ **C. Effects of baseline estimation variance**
- ▶ **D. Compute metrics #rollouts vs. #tokens**

▼ Citation

Please cite this work by:

```
@misc{Cheng2026isocompute,
  author={Cheng, Zhoujun* and Xie, Yutao* and Qu, Yuxiao* and Setlur, /
  title={IsoCompute Playbook: Optimally Scaling Sampling Compute for RL
  howpublished={url{https://compute-optimal-rl-llm-scaling.github.io/}},
  year={2026},
}
```

Copy